BÀI TẬP CÁ NHÂN 3 – VT-002

AI-assisted Development và Quality Engineering

Sinh viên	Nguyễn Duy Tâm – 23001928
Nhóm / vai trò	SE2026-T01 / Thành viên
Đề tài	AI-01 – RAG trợ giảng cho tài liệu môn học
Repository	doandinhlinh25/thanh-tam-tam-linh
Baseline	`8aaceec1c4d4958e69536c622ad78dc58825ad29`
Branch nộp bài	`bt3/23001928-vt002-idempotency`
Ngày kiểm chứng	08/11/2026; Python 3.13.7; Windows

Tôi chịu trách nhiệm về toàn bộ code, test và nội dung bài làm; tôi đã khai báo việc sử dụng AI.

1. Specification, acceptance criteria và invariant

Problem statement. Mất phản hồi hoặc thao tác gửi lại có thể khiến cùng một câu hỏi Chat được xử lý nhiều lần. Người dùng cần gửi lại một yêu cầu logic mà vẫn nhận cùng kết quả, không sinh thêm bản ghi kết quả. Khóa idempotency phải được ràng buộc với phạm vi người dùng và toàn bộ payload có ý nghĩa nghiệp vụ.

Đối chiếu repo thật. Baseline mới đã có Chat Python/FastAPI; không dùng lại mô tả skeleton của BT2. Controller Chat sinh `message_id` mới mỗi lần gọi. `PostgresMessageRepository` mới có constructor, chưa triển khai các phương thức của interface. Vì vậy chọn **executable harness** theo ngoại lệ của đề: module điều phối đặt cạnh Chat, kho dữ liệu và RAG giả được inject; chưa nối endpoint production. Lecture 06/07 chưa được cung cấp, nên bài không gán nội dung cho slide chưa đọc.

ID	Acceptance criterion kiểm thử được
AC1	Cùng tenant/user/course, key và payload chuẩn hóa: replay tuần tự hoặc đồng thời trả cùng kết quả; chỉ một bản ghi được commit.
AC2	Cùng scoped key nhưng khác question, conversation hoặc personal-docs: trả 409; không thêm bản ghi.
AC3	Cùng key ở tenant, user hoặc course khác: xử lý độc lập, không replay nhầm dữ liệu.
AC4	Key ASCII an toàn dài 1–64; question dài 1–4000 ký tự, gồm hai biên. Input sai trả 400 trước khi gọi backend.
AC5	Lỗi trước commit không để lại dòng kết quả; retry sau đó thành công. Mất phản hồi sau commit cho phép replay từ cùng repository.

Non-functional constraints. (N1) Kiểm chứng 200 tác vụ đồng thời và so với p95 ≤ 5.0 giây; tách kết luận mô phỏng khỏi production. (N2) Suite không phụ thuộc network, sleep thời gian thực hoặc thứ tự; mỗi test có store/event loop riêng. (N3) Chỉ thêm lát cắt nhỏ, không thay auth/DB nghiệp vụ, không merge trước review nhóm.

Invariant chính: một scoped key chỉ ánh xạ tới một fingerprint và một kết quả đã commit; mọi replay nhận snapshot độc lập. Repository thật phải bảo vệ unique key và ghi message cùng replay record trong một transaction. Khóa in-memory không đủ cho nhiều process.

2. AI-assisted plan, review và test strategy

Prompt lập kế hoạch (brief tác vụ cho Codex, ghi trước patch):

> Đối chiếu repo hiện tại và chọn lát cắt idempotency nhỏ, chạy được. Trước code, chỉ rõ assumptions, file thay đổi, spec và test RED. Sinh test pairwise, lỗi biên 4000, failure/retry, cancellation; sau patch đo coverage, mutation $>$ thành $\geq$, kiểm tra flaky và 200 tác vụ. Không tạo evidence giả hoặc gọi LLM thật.

Assumptions được công khai. Context tenant/user/course trong harness đã được fixture cấp quyền; không dùng các header demo làm nguồn xác thực. Fake repository mô phỏng `get` và `insert_once` atomic. Operation chỉ tính kết quả, chưa được tự commit side effect ngoài kiểm soát. Key không có TTL trong lát cắt này; vì vậy không có logic phụ thuộc đồng hồ.

Hai điểm đã sửa/bác bỏ trong plan.

1. *Dùng QuotaService Node làm guard chung cho Chat*: bác bỏ. `services/tenant-service/services/quota.service.ts` đã dedupe quota bằng MongoDB, nhưng không lưu/replay response của Chat Python. Giữ ownership của mỗi service; thêm module thuần Python cạnh Chat.
2. *Dùng X-Request-Id như idempotency key*: sửa. Controller hiện coi đây là tracing ID tùy chọn; không ràng buộc với payload và vẫn sinh message ID mới. Harness dùng explicit key riêng, scope tenant/user/course, hash đủ ba trường của request schema.

File dự định thay đổi và trình tự.

1. Lưu plan ở `docs/plans/2026-11-08/vt002-idempotency.md` và nháp AI tests; tạo branch từ baseline đã fetch.
2. Thêm `app/services/idempotency.py` trong Chat cùng test/harness ở `experiments/vt002/`; chạy scaffold ngây thơ để thu RED rồi commit.
3. Patch register pending task, fingerprint, replay repository và snapshot; đọc diff; thu GREEN rồi commit.
4. Tạo nhánh/worktree mutation riêng; chạy suite và lưu diff/log. Đo coverage, chạy năm thứ tự test, chạy load; viết README, review và báo cáo.

Test strategy dựa trên risk.

Mức	Kiểm tra / lý do	Giới hạn
Unit	Validation, fingerprint, exact boundary, pending conflict và snapshot; chạy nhanh, không I/O ngoài.	Không chứng minh transaction DB.
Integration kiểu API	HarnessApi nối facade, coordinator, FakeRag và MemoryRepository; kiểm tra 200/400/409/503, số lần gọi và số dòng.	Là in-process facade; chưa phải FastAPI HTTP.
E2E thật	Hoãn browser/gateway/auth/DB/LLM; bước tiếp theo có giá trị là hai process cùng DB và crash injection.	Không dựng UI hay gọi paid API chỉ để tăng số test.

Không cần trong patch này: test giao diện không thay đổi, gọi internet thật trong mỗi unit test, hoặc lặp happy path chỉ đổi tên test. Negative cases được tách khỏi covering array hợp lệ để có oracle rõ cho 400/409; failure path T15 dùng exception được điều khiển, không dùng network outage ngẫu nhiên.

3. Pairwise/combinatorial testing

Bốn factor nhị phân. A: tenant A/B; B: question dài 1/4000; C: tuần tự/đồng thời; D: personal-docs false/true. Key và scope còn lại hợp lệ. Mỗi case gửi **hai lần cùng scoped key/payload**; các factor thay đổi để kiểm tra replay qua nhiều tổ hợp, không chỉ một happy path.

Full factorial cần $2^4 = 16$ cấu hình. Chọn 8 hàng với ba bit a, b, c và $d = a \oplus b \oplus c$. `pairwise.py` kiểm tra độc lập mọi cặp factor và bốn tổ hợp giá trị: tổng $\binom{4}{2} \times 4 = 24$ cặp cần phủ, không còn cặp thiếu. Đây là phủ strength 2, không phải exhaustive hay bảo đảm tương tác ba/bốn chiều.

Case	A: tenant	B: độ dài	C: lịch gọi	D: personal
T01	A (0)	1 (0)	Tuần tự (0)	False (0)
T02	A (0)	1 (0)	Đồng thời (1)	True (1)
T03	A (0)	4000 (1)	Tuần tự (0)	True (1)
T04	A (0)	4000 (1)	Đồng thời (1)	False (0)
T05	B (1)	1 (0)	Tuần tự (0)	True (1)
T06	B (1)	1 (0)	Đồng thời (1)	False (0)
T07	B (1)	4000 (1)	Tuần tự (0)	False (0)
T08	B (1)	4000 (1)	Đồng thời (1)	True (1)

Test-case contract cho T01–T08. Mỗi dòng sau là một test method riêng, được sinh từ đúng hàng covering array; không chỉ là tám lần ghi cùng một case.

ID	Risk / AC	Level	Precondition / input	Expected result	P
T01	AC1; replay	H	Fresh store; row 0000; gửi hai lần.	Cả hai 200; cùng body/ID; call=1, commit=1.	P0
T02	AC1; race	H	Fresh store; row 0011; gather hai lệnh.	Như T01 dù hai request cùng chờ.	P0
T03	AC1/4	H	Row 0101; question đúng 4000 ký tự.	Nhận hợp lệ; replay cùng body; commit=1.	P0
T04	AC1/4	H	Row 0110; biên 4000 và đồng thời.	200, cùng ID; call=1, commit=1.	P0
T05	AC1	H	Row 1001; tenant B, personal=true.	Replay ổn định trong scope B.	P1
T06	AC1; race	H	Row 1010; tenant B, đồng thời.	200, cùng body; chỉ một dòng.	P0
T07	AC1/4	H	Row 1100; tenant B, dài 4000.	Biên được chấp nhận; replay ổn định.	P0
T08	AC1/4	H	Row 1111; cả bốn mức 1.	200, cùng body; call=1, commit=1.	P0

Chú giải: H = integration qua facade harness; U/H ở trang sau = validation/logic unit quan sát qua cùng facade. P0 ưu tiên cao nhất. Ngoài status, oracle luôn kiểm tra dữ liệu hoặc side effect; vì vậy test có thể bắt một implementation trả 200 nhưng ghi trùng.

Tái lập dữ liệu: input là chuỗi Q lặp theo độ dài, ID tenant/user/course giả và key cố định; không UUID ngẫu nhiên hay dữ liệu cá nhân. Một constructor fixture tạo store mới cho từng test. Các case negative/boundary bổ sung ở trang kế tiếp không làm thay đổi tuyên bố phủ pairwise hợp lệ.

4. Test cases bổ sung theo risk

Fixture chung: scope A hợp lệ, key hợp lệ, FakeRag không có mạng và MemoryRepository ban đầu rỗng, trừ khi ghi khác. File test duy nhất: `experiments/vt002/test_idempotency.py`.

ID	Risk / AC	Level	Precondition / input	Expected result	P
T09	AC2; negative	H	Gửi Q rồi đổi question, giữ key/scope.	409 KEY_CONFLICT; backend=1, commit=1.	P0
T10	AC2; hash	U/H	Lần lượt đổi conversation hoặc personal; thử cả defaults tường minh.	Trường khác gây 409; default null/false tương đương trường bỏ trống.	P0
T11	AC3; isolation	H	Cùng key, lần lượt đổi từng tenant/user/course.	Bốn scope cho bốn kết quả độc lập; bốn dòng.	P0
T12	AC4; negative	U/H	Key null/rỗng/có space/non-ASCII/dài 65; key 64 hợp lệ.	Key sai: 400, không backend/commit; key 64: 200.	P0
T13	AC4; boundary	U/H	Question dài 0, 1, 3999, 4000, 4001.	400, 200, 200, 200, 400; tổng ba dòng.	P0
T14	AC4; schema	U/H	Payload null/list/thiếu Q/sai kiểu/field lạ; scope sai hoặc rỗng.	400 ở mọi input sai; backend=0, commit=0.	P0
T15	AC5; failure	H	Backend fail có điều khiển trước commit; retry cùng request.	Lần đầu 503 và 0 dòng; retry 200; hai call, một commit.	P0
T16	AC5; lost reply	H	Bỏ response đã commit; coordinator mới dùng cùng store.	Replay cùng body/ID; backend không tăng, commit=1.	P0
T17	AC1; 2 owners	H	Hai coordinator cùng key và cùng repository.	Hai phép tính được phép; một dòng commit và cùng response cuối.	P0
T18	Hủy chờ	H	Gate giữ operation; hủy một waiter; thả gate.	Waiter còn lại hoàn thành; một call/commit.	P0
T19	AC1; tải	H	200 caller cùng key; Event giữ operation pending.	200 kết quả giống nhau; backend=1, commit=1.	P0
T20	Shared state	U/H	Sửa input khi pending, rồi sửa citation trong output.	Answer dùng input gốc; replay citation không bị sửa theo caller.	P1
T21	AC2; pending	H	Q còn pending; gửi question khác cùng key.	409 trước commit; original hoàn thành đúng một dòng.	P0

Failure-path gắn flaky risk. T15 không đợi thời điểm network ngắt; fixture ném lỗi ở điểm trước commit đã biết. T18/T19/T21 đồng bộ bằng Event thay vì sleep cố định. Như vậy oracle dựa trên trạng thái đã đạt, không dựa vào giả định "máy sẽ chạy đủ nhanh sau vài mili giây".

AI-generated tests đã sửa/xóa. W1 chỉ assert status=200: thay bằng equality body/ID, call count và commit count trong T01–T08. W2 lặp sequential happy path chỉ đổi tên: xóa bản trùng, dùng lịch concurrent có gate và failure/retry. Nháp và quyết định được lưu trong `ai-draft-tests.txt`; không tăng số case bằng những bản sao cùng oracle/lịch chạy.

5. Test-first change và mutation experiment

Baseline và branch. Fetch repo chính thức trước khi chọn baseline `8aaceec`; tạo nhánh riêng `bt3/23001928-vt002-idempotency`. Không merge. RED là scaffold mới của harness, không phải tuyên bố bộ test production tại baseline đã fail.

Mốc	Commit	Evidence thực thi
RED	`d51ebc0`	20 method; 27 assertion failures và 2 errors ở subtests payload sai; T01 trả message-0001 rồi message-0002.
GREEN code	`1f4fc75`	20/20 pass sau patch. Lưu log, giữ nguyên yêu cầu replay.
Evidence cuối	`3627038`	Thêm T21 cho gap pending conflict: 21/21 pass; coverage và năm thứ tự chạy được lưu.
Mutation tạm	`5aec9fc`	Nhánh/worktree riêng, cha là 1f4fc75; không thuộc ancestry nhánh nộp. Log và diff được chép vào evidence.

Số lỗi RED lớn hơn số method vì unittest đếm từng subtest. Các lỗi duplicate ID và tăng backend call chứng minh fail đúng hành vi; hai lỗi input sai cũng được giữ nguyên trong log, không che đi. Không có import/dependency error làm lý do RED.

Patch nhỏ và review diff. Thêm module 89 dòng, 57 executable statements; không sửa controller/schema production. Luồng chính:

```
scoped_key, fingerprint, snapshot = normalize(scope, key, payload)
async with self._lock:
    # Reuse pending task for same hash; raise 409 on a different hash.
    # Otherwise register _perform(...) before yielding to the backend.
return deepcopy(await asyncio.shield(task))

# _perform: repository.get -> replay OR operation -> insert_once
# finally: remove the finished entry from the pending map.
```

Review kiểm tra: lock không giữ trong lúc chạy backend; hash chứa question/conversation/personal-docs với defaults chuẩn hóa; shield bảo vệ waiter còn lại; snapshot tách mutable state; kết quả hoàn thành nằm ở repository, không bị giữ mãi trong cache. `insert_once` là contract atomic bắt buộc với adapter DB thật. T17 cho thấy vẫn có thể tính hai lần giữa hai coordinator.

Mutation bắt buộc. Schema request thật chấp nhận question dài tối đa 4000, nên guard đúng và mutant là:

```
# Correct (submission branch)
len(question) > MAX_QUESTION_LENGTH
# Deliberate mutant (temporary branch only)
len(question) >= MAX_QUESTION_LENGTH
```

Mutation làm input dài đúng 4000 bị trả 400. Suite tại code commit 1f4fc75 bắt lỗi bằng T03/T04/T07/T08 và T13: **6 assertion/subtest failures thuộc 5 method**; trong T13 có cả sai status và sai tổng số dòng. Child unittest exit=1; wrapper mutation xác nhận đây là assertion failure rồi kết thúc thành công theo contract của công cụ.

Kết luận sức mạnh test: mutant đã chết, không cần sửa test để bắt mutant này. T21 được thêm vì coverage/risk pending conflict, không phải do mutation sống sót. Một mutant bị bắt không chứng minh mọi lỗi idempotency đều bị phát hiện; không suy ra mutation score toàn dự án.

6. Coverage, flaky review và performance experiment

Coverage đúng phạm vi. coverage.py 7.13.1 đo riêng `app/services/idempotency.py`: **57/57 statements, 18/18 branch exits**, không còn missing line/branch. Lần đo trước thiếu dòng 69, tức conflict khi request còn pending; T21 bổ sung đúng tình huống đó. Đây không phải coverage của FastAPI, fake repository, DB hay toàn dự án; cũng không phải condition/MC/DC coverage của mọi vế boolean.

Flaky risk	Cách kiểm soát và evidence	Điều chưa chứng minh
Time	Không TTL/deadline trong business slice; không sleep theo thời gian thực. Event điều khiển điểm pending/failure; sleep(0) chỉ nhường lịch.	Chưa có expiry, timeout hay retry theo đồng hồ production.
Order/state	Store, backend, loop mới mỗi test; replay năm subprocess với seed thứ tự 0–4: 21/21 pass trong cả năm.	Năm permutation không phủ mọi thứ tự hay mọi lịch OS.
Network	Gating tests không mở socket, không gọi LLM/DB ngoài; T15 inject lỗi xác định.	Chưa kiểm tra packet loss, timeout thật, DNS hay proxy.
Concurrency	Gate pending và assert call/commit count; T18/T19/T21.	Một event loop khác multi-process và DB thật.

Môi trường chạy. Python 3.13.7 trên Windows. Sandbox Codex treo cả ví dụ asyncio tối thiểu; các lần chạy ghi log được thực hiện ngoài sandbox với fixture cục bộ. Không coi việc treo môi trường là RED nghiệp vụ. Coverage là công cụ test cài riêng, không thêm runtime dependency vào repo.

Harness tải. `load.py` tạo 200 coroutine đợi start Event; chỉ thả khi cả 200 đã ready. Đo bằng `perf_counter`; latency từng request gồm xử lý sau barrier tới response facade. p95 nearest rank: phần tử thứ 190 trong 200 mẫu đã sort, index 189. Mỗi workload lưu đủ 200 mẫu trong `evidence/load.json`.

Workload	Peak caller	Backend call	Commit	p95 (ms)
200 key khác nhau	200	200	200	4.6169
200 retry cùng key	200	1	1	1.8964

Hai workload đều nhận 200 response thành công, không có mẫu lỗi bị loại khỏi phép đo. Peak operation của fake backend lần lượt là 200 và 1; p50 lần lượt 4.1234 và 1.6459 ms. Cả hai p95 mô phỏng nhỏ hơn **5000 ms = 5.0 s**.

Giới hạn kết luận. FakeRag chỉ yield một lần; store là dict được khóa trong process. Không có HTTP/TLS, xác thực, I/O DB, inference/token generation, tranh tài nguyên nhiều process hay crash. Đây là một lần đo smoke-load trên dữ liệu giả, không có warm-up/statistical confidence interval; không được ghi "hệ thống RAG production đạt SLO". Tải thật tiếp theo cần DB/provider stub có latency điều khiển, HTTP client 200 inflight và metrics lỗi/tail latency.

7. Traceability và engineering review

Quy ước đường dẫn: F = `experiments/vt002/test_idempotency.py`; P = `services/chat-service/app/services/idempotency.py`. Commit C = `3627038`, chứa suite cuối và log; code thực thi của P xuất phát từ `1f4fc75`.

AC / Risk	Test case	File / commit	Kết quả / evidence
AC1 replay	T01–T08, T19	F, P / C	GREEN; cùng body, một commit.
AC1 nhiều owner	T17	F, support.py / C	Một dòng; hai phép tính vẫn được phép.
AC2 hash conflict	T09/T10/T21	F, P / C	409 cả completed và pending; không thêm dòng.
AC3 isolation	T11	F, P / C	Bốn scope độc lập; bốn dòng.
AC4 input/biên	T12–T14	F, P / C	GREEN; exact 4000 hợp lệ.
AC5 retry/lost	T15/T16	F, support.py / C	Precommit fail=0 dòng; retry/replay đúng.
Cancel/state	T18/T20	F, P / C	Không hủy shared work; snapshot độc lập.
Mutation	T03/04/07/08/13	F / 1f4fc75; mutant 5aec9fc	KILLED; mutation.log và mutation.diff.
Order/network	T01–T21	verify.py / C	Năm order seeds pass; không network ngoài.
200 / p95	Hai workload	load.py / C	Peak 200; p95 4.6169/1.8964 ms mô phỏng.

Hai điểm AI-generated code/tests có vẻ hợp lý nhưng chưa đáng tin ngay.

1. Status-only oracle có thể pass dù response ID khác và DB ghi hai dòng. Vì vậy cần kiểm tra equality, số call, số commit và data isolation. Nháp W1/W2 đã được thay/xóa như trang 4 và AI Usage Log.
2. Dictionary + asyncio lock nhìn như "exactly once", nhưng chỉ khóa một loop. T17 cố ý dùng hai coordinator và quan sát hai phép tính/một commit. Muốn bảo vệ giao dịch thật phải có unique constraint và transaction; không hứa chỉ tính phí LLM một lần.

Review actual diff. Không sửa code Node/UI, không đổi production endpoint/auth, không nâng dependency runtime. Patch có snapshot, phân biệt conflict/invalid/backend failure và dọn pending entries. Generated bytecode lỡ vào RED commit đã được bỏ khỏi cây cuối và ignore; lịch sử RED vẫn được giữ nguyên để không viết lại evidence. Review trong bài là Codex tự review, không gán là nhóm hoặc một người độc lập đã duyệt.

Decision: chấp nhận có điều kiện cho *bài tập harness*; CR production còn partial. Điều kiện trước merge: review nhóm, adapter persistence transactional, tích hợp context đã xác thực và kiểm tra hai process/crash. Chưa chạy root npm test/browser E2E vì không sửa các thành phần đó; suite Python được chạy bằng các lệnh ghi ở README, không suy diễn từ test Node.

Evidence truy cập được: README chạy lại; plan; test source; GREEN log; mutation log; coverage; load samples; test/review report.

8. Reflection và cách tái lập

Điều làm tôi tăng confidence là sự liên kết giữa contract, lỗi quan sát được và kiểm tra hồi quy. Log RED cho thấy hai lần gửi nhận message ID khác nhau, nên failure có ý nghĩa hơn lỗi import hoặc thiếu thư viện. Sau patch, test không chỉ đếm response thành công mà còn kiểm tra số bản ghi, số phép tính và snapshot. Mutation ở đúng biên 4000 bị phát hiện giúp xác nhận oracle thực sự phân biệt hành vi đúng với sai.

Tôi cũng nhận ra coverage cao không thay thế phân tích risk. Lần đo đầu thiếu đường xung đột khi request còn pending; bổ sung T21 làm rõ một tình huống nghiệp vụ cụ thể. Năm thứ tự chạy đều pass và việc dùng Event thay sleep tăng confidence về khả năng tái lập. Tuy nhiên, các phép thử vẫn chạy với repository trong bộ nhớ. Chúng chưa chứng minh tính atomic của transaction thật, quyền truy cập qua gateway, khả năng phục hồi khi process chết hoặc chi phí gọi provider bị nhân đôi.

Kết quả tải thấp hơn năm giây chỉ phản ánh fixture mô phỏng. Tôi không thể dùng vài mili giây đó để kết luận hệ thống RAG đáp ứng production SLO. Test tiếp theo có giá trị cao nhất là chạy hai process với cùng database, gửi cùng key, rồi làm process dừng giữa ghi message và lưu replay record. Tôi sẽ yêu cầu chỉ tồn tại một kết quả đã commit, retry trả đúng ID và không rò dữ liệu chéo tenant. Sau đó mới đo tải HTTP với backend giả có độ trễ điều khiển, lưu cả lỗi và tail latency. Cách tiếp cận này biến giới hạn hiện tại thành bước kiểm chứng rõ ràng thay vì dùng AI hoặc phần trăm coverage làm bảo chứng.

Lệnh chạy lại tại gốc repository:

```
python experiments/vt002/run.py --phase green
python experiments/vt002/pairwise.py
python experiments/vt002/verify.py
python experiments/vt002/load.py
# Optional test tooling, not a runtime dependency:
python -m pip install coverage==7.13.1
python experiments/vt002/verify.py
```

RED và mutation: README hướng dẫn tạo scratch worktree tại d51ebc0 hoặc 1f4fc75; mutation diff được apply ở scratch, không apply vào nhánh nộp bài. Wrapper RED/mutation xác nhận child test phải fail bằng assertion. Với GREEN, child exit phải bằng 0.

Nguồn đối chiếu baseline: Chat controller; request schema; Postgres repository; quota service. Hai snapshot cũ/mới được phân biệt bằng commit, không mô tả repo hiện tại chỉ dựa trên bài trước.

Trạng thái nộp: branch phục vụ review, chưa merge. Bản reflection và toàn bộ báo cáo được AI hỗ trợ soạn theo bằng chứng phiên làm việc; sinh viên phải đọc, đối chiếu trải nghiệm và chịu trách nhiệm trước khi nộp.

Tên PDF: `BT3_23001928_VT-002.pdf`.

A. AI Usage Log – không tính giới hạn nội dung chính

Công cụ: Codex trong phiên làm việc 08/11/2026; không gọi API LLM trả phí để tạo dữ liệu test. AI hỗ trợ toàn bộ plan, code, tests, thực nghiệm, review và bản thảo reflection/LaTeX. Phần kiểm chứng do công cụ thực thi tạo log; không coi output văn bản của AI là bằng chứng correctness.

Mục	Prompt / output chính	Giữ / sửa / bác bỏ và evidence
A1	Lập plan idempotency theo repo thật, ghi assumptions trước code.	Giữ harness có ranh giới; bác bỏ dùng Node quota như replay Chat và dùng tracing ID làm key. Plan lưu trước RED commit.
A2	Sinh tests replay, pairwise bốn factor, biên 4000 và failure path.	Giữ covering array 8 hàng; verifier xác nhận 24/24 cặp. W1 status-only sửa bằng equality/count; W2 happy-path trùng bị xóa và thay lịch concurrent/failure.
A3	Tạo patch nhỏ với scoped key, fingerprint, pending task và repository.	Review diff trước GREEN; giữ snapshot/shield; không nối endpoint chưa đủ persistence/auth. Code commit 1f4fc75.
A4	Đổi đúng toán tử biên $>$ thành $\geq$, chạy lại suite.	Giữ mutant chỉ trong nhánh tạm 5aec9fc; T13 và bốn pairwise cases phát hiện; diff/log được lưu.
A5	Đo coverage và tìm missing risk, chạy nhiều thứ tự.	Thêm T21 vì pending conflict chưa chạy; 57/57 statements, 18/18 branches, năm order runs pass. Không diễn giải là 100% correctness.
A6	Đo 200 tác vụ unique/retry và viết báo cáo.	Giữ raw samples; ghi rõ fake RAG/store, p95 chỉ mô phỏng. Reflection là bản thảo AI hỗ trợ, cần sinh viên kiểm tra.

Nháp test yếu và thay đổi (trích).

```
# W1 (weak): assert response["status"] == 200
# Replaced by:
assert first == second
assert backend.calls == 1
assert repository.commits == 1

# W2 (duplicate): run the same sequential replay under another name.
# Removed/replaced with event-gated concurrent replay and failure/retry.
```

Lưu ý provenance: A1–A6 là brief/prompt tác vụ và quyết định của cùng phiên Codex, không phải lời xác nhận người học đã review độc lập. Không sao chép log hay test của thành viên khác; tên file/tài liệu của nhóm chỉ được dùng làm nguồn đối chiếu baseline.

B. Trích log thực thi và định danh đầy đủ

```
RED (d51ebc0):
  exit_code=1
  Ran 20 tests in 0.975s
  FAILED (failures=27, errors=2)
  T01: message-0001 != message-0002

GREEN final (3627038):
  exit_code=0
  Ran 21 tests in 1.296s
  OK

MUTATION (5aec9fc, parent 1f4fc75):
  len(question) > MAX_QUESTION_LENGTH
    -> len(question) >= MAX_QUESTION_LENGTH
  T13 length=4000: AssertionError: 400 != 200
  Ran 20 tests in 1.208s
  FAILED (failures=6)

COVERAGE (coordinator only):
  Stmts=57 Miss=0 Branch=18 BrPart=0 Cover=100%
ORDER:
  seed 0,1,2,3,4 -> exit_code=0 each
PAIRWISE:
  8 rows; 4 binary factors; all 24 factor/value pairs covered
LOAD (p95 nearest rank):
  unique: n=200 peak=200 commits=200 p95_ms=4.6169
  replay: n=200 peak=200 commits=1   p95_ms=1.8964
```

Các commit:

- Baseline: `8aaceec1c4d4958e69536c622ad78dc58825ad29`.
- RED: `d51ebc03bd747a35a428dc40cf6d491541fcdcbf`.
- Code GREEN: `1f4fc75c153864d4bec4e544c65761133b4b306a`.
- Evidence/suite cuối: `362703817e2d01a7915a076b954fcbf3038771b5`.
- Mutant tạm: `5aec9fc3b6b1fcb5c7ef6a2ca49b5017e9c28625` (không nằm trong branch nộp).

Log đầy đủ và mẫu latency nằm trong branch, không dùng ảnh chụp thay code. Các thời gian trên là số đo của phiên đã lưu; chạy lại có thể khác do máy và scheduler. Mọi nội dung về production DB, auth, browser E2E và process crash vẫn là giới hạn được công khai.